DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	YELIGETI ADWAITAN
ROLL NO	AV.SC.U4CSE25254
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	5-9
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	10-11
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	12-14
4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	15-16
5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index	17-22

	5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10)Search data in SLL 11)Display or Traverse 12)Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10)Exit	23-28
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	29-32
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	33-40
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion	41-48

	3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	49-58
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	59-63
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	64-69

WEEK – 1

PROGRAM-1(A): A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!".

CODE:

week_1a.py - C:/Users/adwai/Desktop/DSA/week_1a.py (3.14.0)

File Edit Format Run Options Window Help

```
def Launch(n):
    if n==0:
        print("Launch")
        return
    print(n)
    return Launch(n-1)

n = int(input("Enter Countdown:"))
Launch(n)
```

OUTPUT:

```
>>>
================ RESTART: C:/Users/adwai/Desktop/DSA/week_1a.py ================
Enter Countdown:5
5
4
3
2
1
Launch
>>>
```

PROGRAM-1(B): Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years.

CODE:

```
week_1b.py - C:/Users/adwai/Desktop/DSA/week_1b.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
def compound(power,n):
    if n==1:
        return power
    return power*compound(power,n-1)

n = int(input("Enter number of years:"))
p = int(input("Enter principle growth:"))
res =  compound(p,n)
print(res)
```

OUTPUT:

```
>>>
================ RESTART: C:/Users/adwai/Desktop/DSA/week_1b.py ================
Enter number of years:5
Enter principle growth:5
3125
>>>
```

PROGRAM-1(C): A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

CODE:

week_1c.py - C:/Users/adwai/Desktop/DSA/week_1c.py (3.14.0)

File Edit Format Run Options Window Help

```
def search(arr,i,val):
    if i>len(arr):
        return -1
    if arr[i]==val:
        return i
    else:
        return search(arr,i+1,val)
n = int(input("Enter size of an array:"))
arr = []
for i in range(n):
    num = int(input("Enter Id:"))
    arr.append(num)
val = int(input("Enter Id to search:"))
res = search(arr,0,val)
if res != -1:
    print("Id found at index:",res)
else:
    print("Id Not found")
```

OUTPUT:

```
>>>
================ RESTART: C:/Users/adwai/Desktop/DSA/week_1c.py ================
Enter size of an array:5
Enter Id:1
Enter Id:2
Enter Id:3
Enter Id:4
Enter Id:5
Enter Id to search:4
Id found at index: 3
>>>
```

PROGRAM-1(D):). In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

CODE:

week_1d.py - C:/Users/adwai/Desktop/DSA/week_1d.py (3.14.0)

File Edit Format Run Options Window Help

```
def fact(n):
    if n==0:
        return 1
    else:
        return n*fact(n-1)
n = int(input("Enter value:"))
res = fact(n)
print("Factorial is :",res)
```

OUTPUT:

```
>>>
================ RESTART: C:/Users/adwai/Desktop/DSA/week_1d.py ================
Enter value:5
Factorial is : 120
>>>
```

PROGRAM-1(E): Write a recursive python program to print the first n terms of the Fibonacci series.

CODE:

week_1e.py - C:/Users/adwai/Desktop/DSA/week_1e.py (3.14.0)

File Edit Format Run Options Window Help

```
def fib(n):
    if n==0:
        return 0
    if n==1:
        return 1
    return fib(n-1)+fib(n-2)

n = int(input("enter value:"))
for i in range(n):
    print(fib(i),end=" ")
```

OUTPUT:

```
>>>
================ RESTART: C:/Users/adwai/Desktop/DSA/week_1e.py ================
enter value:10
0 1 1 2 3 5 8 13 21 34
>>>
```

WEEK – 2

PROGRAM-2(A): Write a python program to implement linear search.

CODE:

```
week_2a.py - C:/Users/adwai/Desktop/DSA/week_2a.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
def linearSearch(arr,val):
    for i in range(len(arr)):
        if arr[i] == val:
            return i
    return -1

n = int(input("Enter no of elements:"))
arr = []
for i in range(n):
    num = int(input("Enter value:"))
    arr.append(num)
val = int(input("Enter number to search:"))
res = linearSearch(arr,val)
if res!=-1:
    print("Element found at the index:",res)
else:
    print("Element not found")
```

OUTPUT:

```
================ RESTART: C:/Users/adwai/Desktop/DSA/week_2a.py ================
Enter no of elements:5
Enter value:1
Enter value:2
Enter value:3
Enter value:4
Enter value:5
Enter number to search:3
Element found at the index: 2
```

PROGRAM-2(B): Write a python program to implement binary search with and without sorted array as an input.

CODE:

```
week_2b.py - C:/Users/adwai/Desktop/DSA/week_2b.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
def binarySearch(arr,key):
    low = 0
    high = len(arr)-1
    while low<=high:
        mid = (low+high)//2
        if arr[mid]==key:
            return mid
        if key<arr[mid]:
            high = mid-1
        else:
            low = mid+1
    return -1
n = int(input("Enter no of elements:"))
arr = []
for i in range(n):
    num = int(input("Enter value:"))
    arr.append(num)
val = int(input("Enter number to search:"))
arr.sort()
res = binarySearch(arr,val)
if res!=-1:
    print("Element found at the index:",res)
else:
    print("Element not found")
```

OUTPUT:

```
================ RESTART: C:/Users/adwai/Desktop/DSA/week_2b.py ================
Enter no of elements:5
Enter value:1
Enter value:3
Enter value:5
Enter value:2
Enter value:4
Enter number to search:3
Element found at the index: 2
```

WEEK – 3

PROGRAM-3(A): Write a python program to implement bubble sort.

CODE:

week_3a.py - C:/Users/adwai/Desktop/DSA/week_3a.py (3.14.0)

File Edit Format Run Options Window Help

```
def bubbleSort(arr):
    for i in range(len(arr)):
        for j in range(len(arr)-1-i):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]

n = int(input("Enter no of elements:"))
arr = []
for i in range(n):
    num = int(input("Enter value:"))
    arr.append(num)
bubbleSort(arr)
print(arr)
```

OUTPUT:

```
================ RESTART: C:/Users/adwai/Desktop/DSA/week_3a.py ================
Enter no of elements:5
Enter value:4
Enter value:2
Enter value:5
Enter value:1
Enter value:3
[1, 2, 3, 4, 5]
```

PROGRAM-3(B): Write a python program to implement insertion sort.

CODE:

week_3b.py - C:/Users/adwai/Desktop/DSA/week_3b.py (3.14.0)

File Edit Format Run Options Window Help

```
def insertionSort(arr):
    for i in range(1,len(arr)):
        key = arr[i]
        j = i-1
        while j>=0 and arr[j]>key:
            arr[j+1] = arr[j]
            j=j-1
        arr[j+1] = key

n = int(input("Enter no of elements:"))
arr = []
for i in range(n):
    num = int(input("Enter value:"))
    arr.append(num)
insertionSort(arr)
print(arr)
```

OUTPUT:

```
================ RESTART: C:/Users/adwai/Desktop/DSA/week_3b.py ================
Enter no of elements:5
Enter value:3
Enter value:5
Enter value:1
Enter value:4
Enter value:2
[1, 2, 3, 4, 5]
```

PROGRAM-3(C): Write a python program to implement selection sort.

CODE:

```
week_3c.py - C:/Users/adwai/Desktop/DSA/week_3c.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
def selectionSort(arr):
    for i in range(len(arr)):
        min_index = i
        for j in range(i+1,len(arr)):
            if arr[j]<arr[min_index]:
                min_index = j
        arr[i],arr[min_index]=arr[min_index],arr[i]


n = int(input("Enter no of elements:"))
arr = []
for i in range(n):
    num = int(input("Enter value:"))
    arr.append(num)
selectionSort(arr)
print(arr)
```

OUTPUT:

```
================ RESTART: C:/Users/adwai/Desktop/DSA/week_3c.py ================
Enter no of elements:5
Enter value:2
Enter value:5
Enter value:1
Enter value:4
Enter value:3
[1, 2, 3, 4, 5]
>
```

WEEK – 4

PROGRAM-4(A): Write a python program to implement quick sort.

CODE:

week_4a.py - C:/Users/adwai/Desktop/DSA/week_4a.py (3.14.0)

File Edit Format Run Options Window Help

```
def quicksort(arr,low,high):
    if low<high:
        pivot = low
        i = low
        j = high
        while i<j:
            while arr[i]<=arr[pivot] and i<j:
                i+=1
            while arr[j]>arr[pivot]:
                j-=1
            if i<j:
                arr[i],arr[j]=arr[j],arr[i]
        arr[j],arr[pivot] = arr[pivot],arr[j]
        quicksort(arr,low,j-1)
        quicksort(arr,j+1,high)
n = int(input("Enter no of elements:"))
arr = []
for i in range(n):
    num = int(input("Enter value:"))
    arr.append(num)
quicksort(arr,0,n-1)
print(arr)
```

OUTPUT:

```
================ RESTART: C:/Users/adwai/Desktop/DSA/week_4a.py ================
Enter no of elements:5
Enter value:5
Enter value:3
Enter value:1
Enter value:2
Enter value:4
[1, 2, 3, 4, 5]
```

PROGRAM-4(B): Write a python program to implement merge sort.

CODE:

week_4b.py - C:/Users/adwai/Desktop/DSA/week_4b.py (3.14.0)

File Edit Format Run Options Window Help

```
def mergesort(arr):
    if len(arr)>1:
        mid = len(arr)//2
        left = arr[:mid]
        right = arr[mid:]
        mergesort(left)
        mergesort(right)
        i=j=k=0
        while i<len(left) and j<len(right):
            if left[i]<right[j]:
                arr[k]=left[i]
                k+=1
                i+=1
            else:
                arr[k]=right[j]
                j+=1
                k+=1
        while i<len(left):
            arr[k]=left[i]
            i+=1
            k+=1
        while j<len(right):
            arr[k]=right[j]
            j+=1
            k+=1
n = int(input("Enter no of elements:"))
arr = []
for i in range(n):
    num = int(input("Enter value:"))
    arr.append(num)
mergesort(arr)
print(arr)
```

OUTPUT:

```
================ RESTART: C:/Users/adwai/Desktop/DSA/week_4b.py ================
Enter no of elements:5
Enter value:2
Enter value:3
Enter value:4
Enter value:1
Enter value:5
[1, 2, 3, 4, 5]
```

WEEK – 5

PROGRAM-5: Write a python program to implement a Singly Linked List with the below operations.

CODE:

```
class Node:
    def __init__(self,data):
        self.next = None
        self.data = data
class SLL:
    def __init__(self):
        self.head =  None
    def insert_begin(self,data):
        x = Node(data)
        x.next = self.head
        self.head = x
    def insert_end(self,data):
        x = Node(data)
        if self.head is None:
            self.head = x
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = x
    def delete_begin(self):
        if self.head is None:
```

```
            print("No data to delete")
        else:
            self.head = self.head.next
    def delete_end(self):
        if self.head is None:
            print("No data to delete")
        elif self.head.next is None:
            self.head = None
        else:
            temp = self.head
            while temp.next.next:
                temp = temp.next
            temp.next = None
    def display(self):
        if self.head is None:
            print("No data to display")
        else:
            temp = self.head
            while temp:
                print(temp.data,end="->")
                temp = temp.next
            print(None)
    def insert(self,data,ind):
        x = Node(data)
        if ind==0:
            return self.insert_begin(data)
```

```
        else:
            temp = self.head
            for i in range(ind-1):
                if temp is None:
                    print("Enter valid index")
                    return
                temp = temp.next
            if temp is None:
                print("Enter valid index")
            else:
                x.next = temp.next
                temp.next = x
    def delete(self,data):
        if self.head is None:
            print("List is empty")
            return
        if self.head.data == data:
            return self.delete_begin()
        temp = self.head
        while temp.next and temp.next.data!=data:
            temp = temp.next
        if temp.next is None:
            print("Element not found")
        else:
            temp.next = temp.next.next
    def count(self):
```

```
        if self.head is None:
            print("No data to display")
        else:
            c=0
            temp = self.head
            while temp:
                c+=1
                temp = temp.next
            return c

my_list = SLL()
while True:
    print("\n--- Singly Linked List Menu ---")
    print("1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Insert at Index")
    print("4. Delete from Beginning")
    print("5. Delete from End")
    print("6. Delete by Value")
    print("7. Count Elements")
    print("8. Display List")
    print("9. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        val = int(input("Enter data to insert: "))
        my_list.insert_begin(val)
```

```
    elif choice == 2:
        val = int(input("Enter data to insert: "))
        my_list.insert_end(val)
    elif choice == 3:
        val = int(input("Enter data to insert: "))
        ind = int(input("Enter index: "))
        my_list.insert(val, ind)
    elif choice == 4:
        my_list.delete_begin()
    elif choice == 5:
        my_list.delete_end()
    elif choice == 6:
        val = int(input("Enter value to delete: "))
        my_list.delete(val)
    elif choice == 7:
        c = my_list.count()
        if c > 0:
            print("Total elements:", c)
    elif choice == 8:
        my_list.display()
    elif choice == 9:
        print("Exiting program...")
        break
    else:
        print("Invalid choice! Please try again.")
```

OUTPUT:

```
>>>
================== RESTART: C:/Users/adwai/Desktop/DSA/week_5.py ================

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Index
4. Delete from Beginning
5. Delete from End
6. Delete by Value
7. Count Elements
8. Display List
9. Exit
Enter your choice: 1
Enter data to insert: 2

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Index
4. Delete from Beginning
5. Delete from End
6. Delete by Value
7. Count Elements
8. Display List
9. Exit
Enter your choice: 2
Enter data to insert: 3
```

```
Total elements: 2

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Index
4. Delete from Beginning
5. Delete from End
6. Delete by Value
7. Count Elements
8. Display List
9. Exit
Enter your choice: 4

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Index
4. Delete from Beginning
5. Delete from End
6. Delete by Value
7. Count Elements
8. Display List
9. Exit
Enter your choice: 8
3->None
```

WEEK – 6

PROGRAM-6: Write a python program to implement a Double Linked List with the below operations.

CODE:

week_6.py - C:\Users\adwai\Desktop\DSA\week_6.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self,data):
        self.data = data
        self.next = None
        self.prev = None

class DLL:
    def __init__(self):
        self.head = None
        self.tail = None

    def insert_begin(self,data):
        x = Node(data)
        if self.head is None:
            self.head = self.tail = x
        else:
            x.next = self.head
            self.head.prev = x
            self.head = x

    def insert_end(self,data):
        x = Node(data)
        if self.head is None:
            self.head = self.tail = x
        else:
            x.prev = self.tail
            self.tail.next = x
            self.tail = x

    def insert(self,data,ind):
        if ind < 0:
            print("Enter valid index")
            return
        if ind == 0:
            return self.insert_begin(data)
        x = Node(data)
        temp = self.head
        for i in range(ind-1):
            if temp is None:
                print("Enter valid index")
                return
```

week_6.py - C:\Users\adwai\Desktop\DSA\week_6.py (3.14.0)

File Edit Format Run Options Window Help

```
                return
            temp = temp.next
        if temp is None:
            print("Enter valid index")
            return
        x.prev = temp
        x.next = temp.next
        if temp.next is not None:
            temp.next.prev = x
        else:
            self.tail = x
        temp.next = x

    def delete_begin(self):
        if self.head is None:
            print("List is empty")
        elif self.head == self.tail:
            self.head = self.tail = None
        else:
            self.head = self.head.next
            self.head.prev = None

    def delete_end(self):
        if self.head is None:
            print("List is empty")
        elif self.head == self.tail:
            self.head = self.tail = None
        else:
            self.tail = self.tail.prev
            self.tail.next = None

    def delete_val(self,val):
        temp = self.head
        while temp is not None:
            if temp.data == val:
                if temp == self.head:
                    return self.delete_begin()
                elif temp == self.tail:
                    return self.delete_end()
                else:
                    temp.prev.next = temp.next
```

week_6.py - C:\Users\adwai\Desktop\DSA\week_6.py (3.14.0)

File Edit Format Run Options Window Help

```
                    temp.prev.next = temp.next
                    temp.next.prev = temp.prev
                    return
            temp = temp.next
        print("Value not found")

    def count(self):
        c = 0
        temp = self.head
        while temp is not None:
            c += 1
            temp = temp.next
        print("Count =",c)

    def display(self):
        temp = self.head
        if temp is None:
            print("List is empty")
            return
        while temp is not None:
            print(temp.data,end=" <-> ")
            temp = temp.next
        print("None")

    def display_reverse(self):
        temp = self.tail
        if temp is None:
            print("List is empty")
            return
        while temp is not None:
            print(temp.data,end=" <-> ")
            temp = temp.prev
        print("None")

d = DLL()

while True:
    print("\n1.Insert begin")
    print("2.Insert end")
    print("3.Insert at index")
    print("4.Delete begin")
```

week_6.py - C:\Users\adwai\Desktop\DSA\week_6.py (3.14.0)

File Edit Format Run Options Window Help

```
while True:
    print("\n1.Insert begin")
    print("2.Insert end")
    print("3.Insert at index")
    print("4.Delete begin")
    print("5.Delete end")
    print("6.Delete by value")
    print("7.Count")
    print("8.Display")
    print("9.Display reverse")
    print("10.Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        data = int(input("Enter data: "))
        d.insert_begin(data)
    elif ch == 2:
        data = int(input("Enter data: "))
        d.insert_end(data)
    elif ch == 3:
        data = int(input("Enter data: "))
        ind = int(input("Enter index: "))
        d.insert(data,ind)
    elif ch == 4:
        d.delete_begin()
    elif ch == 5:
        d.delete_end()
    elif ch == 6:
        val = int(input("Enter value: "))
        d.delete_val(val)
    elif ch == 7:
        d.count()
    elif ch == 8:
        d.display()
    elif ch == 9:
        d.display_reverse()
    elif ch == 10:
        break
    else:
        print("Invalid choice")
```

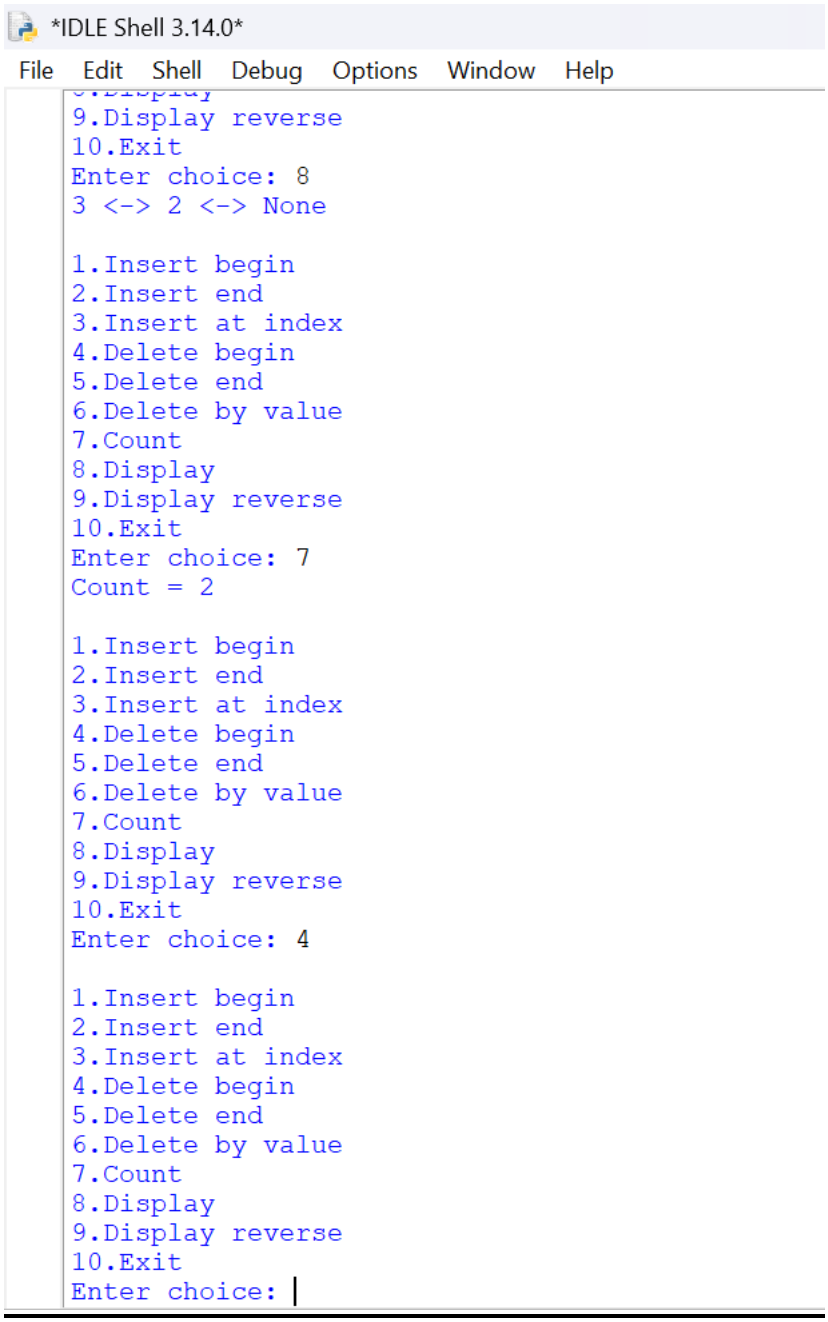
OUTPUT:

```
================= RESTART: C:\Users\adwai\Desktop\DSA\week_6.py ================

1.Insert begin
2.Insert end
3.Insert at index
4.Delete begin
5.Delete end
6.Delete by value
7.Count
8.Display
9.Display reverse
10.Exit
Enter choice: 1
Enter data: 2

1.Insert begin
2.Insert end
3.Insert at index
4.Delete begin
5.Delete end
6.Delete by value
7.Count
8.Display
9.Display reverse
10.Exit
Enter choice: 1
Enter data: 3

1.Insert begin
2.Insert end
3.Insert at index
4.Delete begin
5.Delete end
6.Delete by value
7.Count
8.Display
9.Display reverse
10.Exit
```


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
9.Display reverse
10.Exit
Enter choice: 8
3 <-> 2 <-> None
1.Insert begin
2.Insert end
3.Insert at index
4.Delete begin
5.Delete end
6.Delete by value
7.Count
8.Display
9.Display reverse
10.Exit
Enter choice: 7
Count = 2
1.Insert begin
2.Insert end
3.Insert at index
4.Delete begin
5.Delete end
6.Delete by value
7.Count
8.Display
9.Display reverse
10.Exit
Enter choice: 4
1.Insert begin
2.Insert end
3.Insert at index
4.Delete begin
5.Delete end
6.Delete by value
7.Count
8.Display
9.Display reverse
10.Exit
Enter choice:

WEEK – 7

PROGRAM-7(A): Write a python program to implement stack operations such as push, pop, peek, display using an array.

CODE:

week_7a.py - C:/Users/adwai/Desktop/DSA/week_7a.py (3.14.0)

File Edit Format Run Options Window Help

```
class Stack:
    def __init__(self):
        self.stack = []
    def push(self,data):
        self.stack.append(data)
    def pop(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Deleted element:",self.stack.pop())
    def peek(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Top element:",self.stack[-1])
    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print(self.stack[::-1])
s = Stack()
while True:
    print("\n1.Push")
    print("2.Pop")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        data = int(input("Enter element: "))
        s.push(data)
    elif ch == 2:
        s.pop()
    elif ch == 3:
        s.peek()
    elif ch == 4:
        s.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
================ RESTART: C:/Users/adwai/Desktop/DSA/week_7a.py ================

1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter element: 2

1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter element: 3

1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 4
[3, 2]

1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 2
```

PROGRAM-7(B): Write a python program to implement stack operations such as push, pop, peek, display using a linked list.

CODE:

untitled

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self,data):
        self.data = data
        self.next = None
class Stack:
    def __init__(self):
        self.top = None
    def push(self,data):
        x = Node(data)
        x.next = self.top
        self.top = x
    def pop(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Deleted element:",self.top.data)
            self.top = self.top.next
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:",self.top.data)

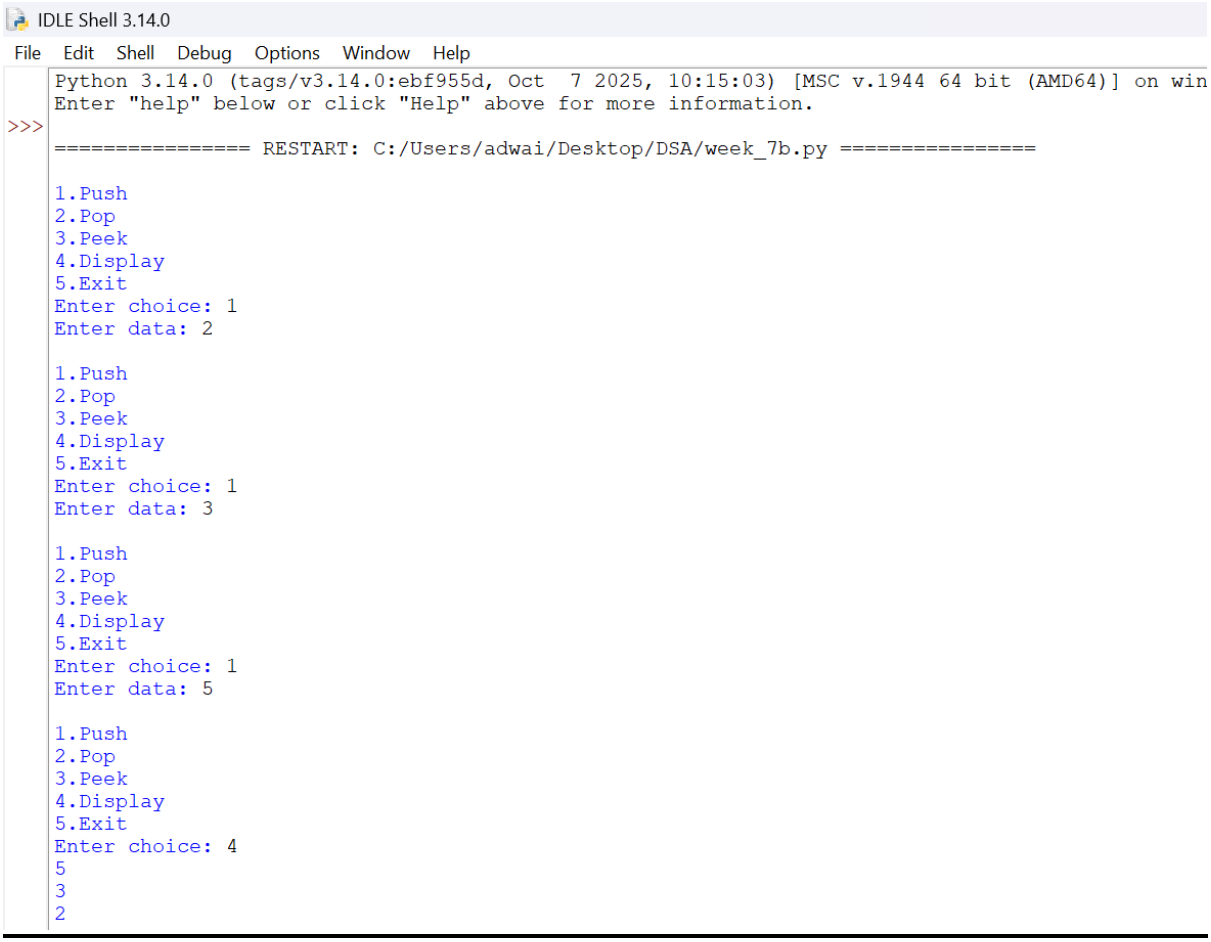
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            while temp is not None:
                print(temp.data)
                temp = temp.next
```

```
s = Stack()
while True:
    print("\n1.Push")
    print("2.Pop")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        data = int(input("Enter data: "))
        s.push(data)
    elif ch == 2:
        s.pop()
    elif ch == 3:
        s.peek()
    elif ch == 4:
        s.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:




```
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 2
Deleted element: 5

1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 4
3
2

1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 5
```

WEEK – 8

PROGRAM-8(A): Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

CODE:

week_8a.py - C:/Users/adwai/Desktop/DSA/week_8a.py (3.14.0)

File Edit Format Run Options Window Help

```
class Queue:
    def __init__(self):
        self.queue = []
    def enqueue(self,data):
        self.queue.append(data)
    def dequeue(self):
        if len(self.queue) == 0:
            print("Queue is empty")
        else:
            print("Deleted element:",self.queue.pop(0))
    def peek(self):
        if len(self.queue) == 0:
            print("Queue is empty")
        else:
            print("Front element:",self.queue[0])
    def display(self):
        if len(self.queue) == 0:
            print("Queue is empty")
        else:
            print(self.queue)
q = Queue()
while True:
    print("\n1.Enqueue")
    print("2.Dequeue")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        data = int(input("Enter data: "))
        q.enqueue(data)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
    elif ch == 5:
        break
```

week_8a2.py - C:/Users/adwai/Desktop/DSA/week_8a2.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self,data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self,data):
        x = Node(data)
        if self.rear is None:
            self.front = self.rear = x
        else:
            self.rear.next = x
            self.rear = x

    def dequeue(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Deleted element:",self.front.data)
            self.front = self.front.next
            if self.front is None:
                self.rear = None

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:",self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            temp = self.front
            while temp is not None:
                print(temp.data,end=" -> ")
                temp = temp.next
```

week_8a2.py - C:/Users/adwai/Desktop/DSA/week_8a2.py (3.14.0)

File Edit Format Run Options Window Help

```
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:",self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            temp = self.front
            while temp is not None:
                print(temp.data,end=" -> ")
                temp = temp.next
            print("None")

q = Queue()

while True:
    print("\n1.Enqueue")
    print("2.Dequeue")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        data = int(input("Enter data: "))
        q.enqueue(data)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
================ RESTART: C:/Users/adwai/Desktop/DSA/week_8a.py ================

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter data: 3

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 2
Deleted element: 3

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 4
Queue is empty

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 5
```

```
================ RESTART: C:\Users\adwai\Desktop\DSA\week_8a2.py ===============

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter data: 2

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 3
Front element: 2

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 2
Deleted element: 2

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 4
Queue is empty

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 5
```

PROGRAM-8(B):

Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

CODE:

week_8b.py - C:/Users/adwai/Desktop/DSA/week_8b.py (3.14.0)

File Edit Format Run Options Window Help

```
class CQueue:
    def __init__(self,size):
        self.size = size
        self.queue = [None]*size
        self.front = -1
        self.rear = -1

    def enqueue(self,data):
        if (self.rear+1)%self.size == self.front:
            print("Queue is full")
        elif self.front == -1:
            self.front = self.rear = 0
            self.queue[self.rear] = data
        else:
            self.rear = (self.rear+1)%self.size
            self.queue[self.rear] = data

    def dequeue(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Deleted element:",self.queue[self.front])
            if self.front == self.rear:
                self.front = self.rear = -1
            else:
                self.front = (self.front+1)%self.size

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:",self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            i = self.front
            while True:
                print(self.queue[i],end=" ")
                if i == self.rear:
```

week_8b.py - C:/Users/adwai/Desktop/DSA/week_8b.py (3.14.0)

File Edit Format Run Options Window Help

```
            print("Queue is empty")
        else:
            print("Front element:",self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            i = self.front
            while True:
                print(self.queue[i],end=" ")
                if i == self.rear:
                    break
                i = (i+1)%self.size
            print()

size = int(input("Enter queue size: "))
q = CQueue(size)

while True:
    print("\n1.Enqueue")
    print("2.Dequeue")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        data = int(input("Enter data: "))
        q.enqueue(data)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Enter queue size: 5
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter data: 2
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 3
Front element: 2
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 3
Front element: 2
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 4
2
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter choice: 5

WEEK – 9

PROGRAM-9: Binary Search Tree (BST)

CODE:

```
class Node:
    def __init__(self,data):
        self.data = data
        self.left = None
        self.right = None


class BST:
    def __init__(self):
        self.root = None


    def insert(self,root,data):
        if root is None:
            return Node(data)
        if data < root.data:
            root.left = self.insert(root.left,data)
        elif data > root.data:
            root.right = self.insert(root.right,data)
        return root


    def delete(self,root,data):
        if root is None:
            print("Value not found")
            return root
```

```
    if data < root.data:
        root.left = self.delete(root.left,data)
    elif data > root.data:
        root.right = self.delete(root.right,data)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        temp = root.right
        while temp.left is not None:
            temp = temp.left
        root.data = temp.data
        root.right = self.delete(root.right,temp.data)
    return root

def inorder(self,root):
    if root is not None:
        self.inorder(root.left)
        print(root.data,end=" ")
        self.inorder(root.right)

def preorder(self,root):
    if root is not None:
        print(root.data,end=" ")
        self.preorder(root.left)
```

```
        self.preorder(root.right)

    def postorder(self,root):
        if root is not None:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data,end=" ")

    def count_leaf(self,root):
        if root is None:
            return 0
        if root.left is None and root.right is None:
            return 1
        return self.count_leaf(root.left)+self.count_leaf(root.right)

    def count_internal(self,root):
        if root is None or (root.left is None and root.right is None):
            return 0
        return 1+self.count_internal(root.left)+self.count_internal(root.right)

    def height(self,root):
        if root is None:
            return 0
        return 1+max(self.height(root.left),self.height(root.right))
```

```
b = BST()

while True:
    print("\n1.Insert")
    print("2.Delete")
    print("3.Inorder")
    print("4.Preorder")
    print("5.Postorder")
    print("6.Count Leaf Nodes")
    print("7.Count Internal Nodes")
    print("8.Height")
    print("9.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        data = int(input("Enter data: "))
        b.root = b.insert(b.root,data)

    elif ch == 2:
        data = int(input("Enter data: "))
        b.root = b.delete(b.root,data)

    elif ch == 3:
        b.inorder(b.root)
        print()
```

```
    elif ch == 4:
        b.preorder(b.root)
        print()

    elif ch == 5:
        b.postorder(b.root)
        print()

    elif ch == 6:
        print("Leaf Nodes:",b.count_leaf(b.root))

    elif ch == 7:
        print("Internal Nodes:",b.count_internal(b.root))

    elif ch == 8:
        print("Height of Tree:",b.height(b.root))

    elif ch == 9:
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
================ RESTART: C:/Users/adwai/Desktop/DSA/week_9a.py ================

1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height
9.Exit
Enter choice: 1
Enter data: 2

1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height
9.Exit
Enter choice: 1
Enter data: 3

1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height
9.Exit
Enter choice: 1
Enter data: 1
```

```
1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height
9.Exit
Enter choice: 1
Enter data: 1

1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height
9.Exit
Enter choice: 1
Enter data: 5

1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height
9.Exit
Enter choice: 1
Enter data: 6
```


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
7.Count Internal Nodes
8.Height
9.Exit
Enter choice: 4
2 1 3 5 6 7
1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height
9.Exit
Enter choice: 5
1 7 6 5 3 2
1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height
9.Exit
Enter choice: 8
Height of Tree: 5
1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height
9.Exit
Enter choice: 9
>>>

WEEK – 10

PROGRAM-10: AVL Tree

CODE:

```
class Node:
    def __init__(self,data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1


class AVL:
    def height(self,root):
        if root is None:
            return 0
        return root.height

    def balance(self,root):
        if root is None:
            return 0
        return self.height(root.left)-self.height(root.right)

    def right_rotate(self,y):
        x = y.left
        t = x.right


        x.right = y
```

```
        y.left = t

        y.height = 1+max(self.height(y.left),self.height(y.right))
        x.height = 1+max(self.height(x.left),self.height(x.right))

        return x

    def left_rotate(self,x):
        y = x.right
        t = y.left

        y.left = x
        x.right = t

        x.height = 1+max(self.height(x.left),self.height(x.right))
        y.height = 1+max(self.height(y.left),self.height(y.right))

        return y

    def insert(self,root,data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left,data)
        elif data > root.data:
```

```
            root.right = self.insert(root.right,data)
        else:
            return root

        root.height = 1+max(self.height(root.left),self.height(root.right))

        b = self.balance(root)

        if b > 1 and data < root.left.data:
            return self.right_rotate(root)

        if b < -1 and data > root.right.data:
            return self.left_rotate(root)

        if b > 1 and data > root.left.data:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if b < -1 and data < root.right.data:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    def min_value(self,root):
        temp = root
```

```
    while temp.left is not None:
        temp = temp.left
    return temp

def delete(self,root,data):
    if root is None:
        return root

    if data < root.data:
        root.left = self.delete(root.left,data)
    elif data > root.data:
        root.right = self.delete(root.right,data)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        temp = self.min_value(root.right)
        root.data = temp.data
        root.right = self.delete(root.right,temp.data)

    root.height = 1+max(self.height(root.left),self.height(root.right))

    b = self.balance(root)
```

```
        if b > 1 and self.balance(root.left) >= 0:
            return self.right_rotate(root)

        if b > 1 and self.balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if b < -1 and self.balance(root.right) <= 0:
            return self.left_rotate(root)

        if b < -1 and self.balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    def inorder(self,root):
        if root is not None:
            self.inorder(root.left)
            print(root.data,end=" ")
            self.inorder(root.right)

    def preorder(self,root):
        if root is not None:
            print(root.data,end=" ")
            self.preorder(root.left)
```

```
        self.preorder(root.right)

    def postorder(self,root):
        if root is not None:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data,end=" ")

    def count_leaf(self,root):
        if root is None:
            return 0
        if root.left is None and root.right is None:
            return 1
        return self.count_leaf(root.left)+self.count_leaf(root.right)

    def count_internal(self,root):
        if root is None or (root.left is None and root.right is None):
            return 0
        return 1+self.count_internal(root.left)+self.count_internal(root.right)

    def tree_height(self,root):
        if root is None:
            return 0
        return 1+max(self.tree_height(root.left),self.tree_height(root.right))
```

```
a = AVL()
root = None

while True:
    print("\n1.Insert")
    print("2.Delete")
    print("3.Inorder")
    print("4.Preorder")
    print("5.Postorder")
    print("6.Count Leaf Nodes")
    print("7.Count Internal Nodes")
    print("8.Height of Tree")
    print("9.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        data = int(input("Enter data: "))
        root = a.insert(root,data)

    elif ch == 2:
        data = int(input("Enter data: "))
        root = a.delete(root,data)

    elif ch == 3:
        a.inorder(root)
```

```
        print()

    elif ch == 4:
        a.preorder(root)
        print()

    elif ch == 5:
        a.postorder(root)
        print()

    elif ch == 6:
        print("Leaf Nodes:",a.count_leaf(root))

    elif ch == 7:
        print("Internal Nodes:",a.count_internal(root))

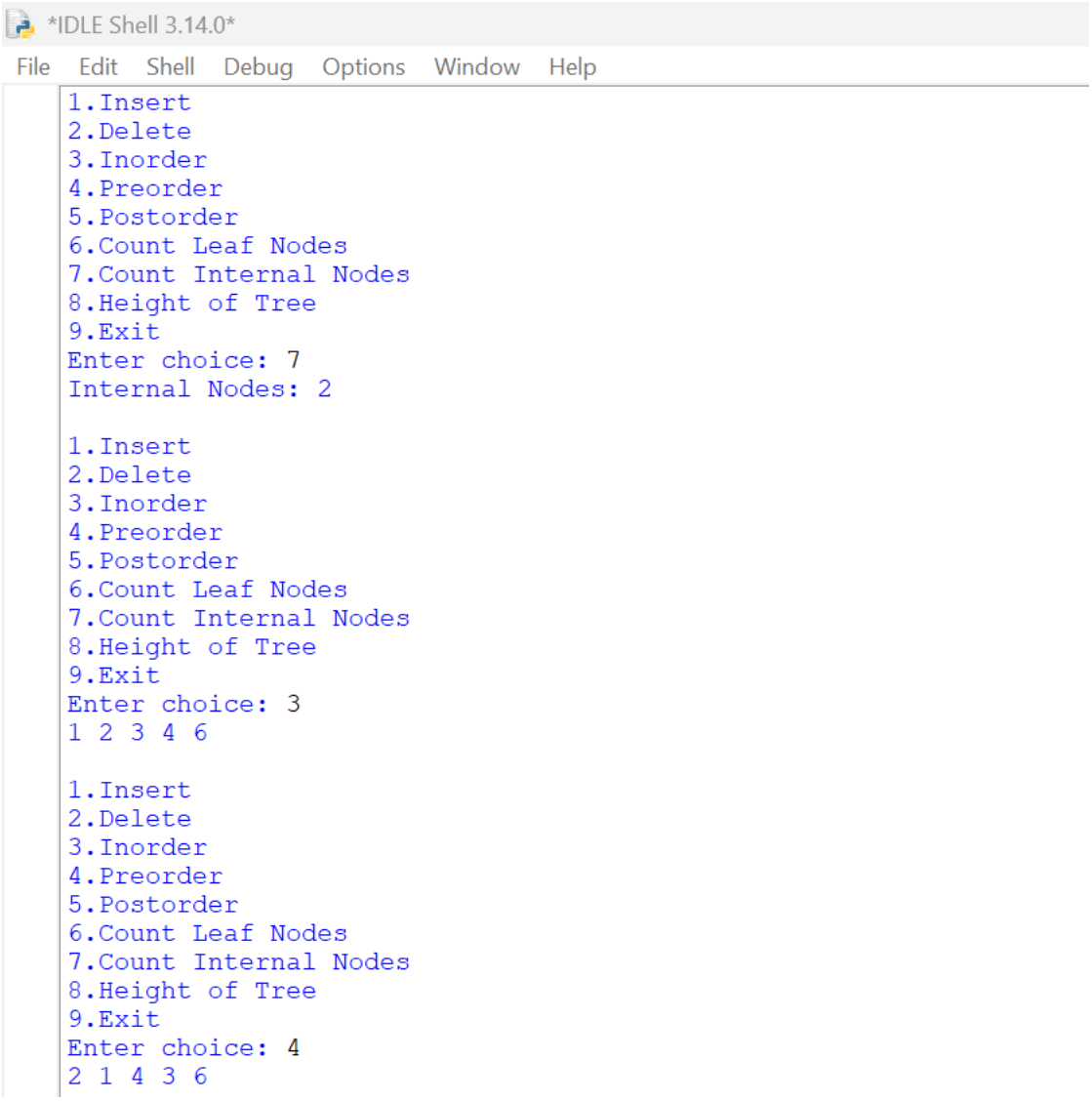
    elif ch == 8:
        print("Height of Tree:",a.tree_height(root))

    elif ch == 9:
        break

    else:
        print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on
Enter "help" below or click "Help" above for more information.
>>>
================ RESTART: C:/Users/adwai/Desktop/DSA/week_10.py ================
1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height of Tree
9.Exit
Enter choice: 1
Enter data: 2
1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height of Tree
9.Exit
Enter choice: 1
Enter data: 1
1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height of Tree
9.Exit
Enter choice: 1
Enter data: 4


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height of Tree
9.Exit
Enter choice: 7
Internal Nodes: 2
1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height of Tree
9.Exit
Enter choice: 3
1 2 3 4 6
1.Insert
2.Delete
3.Inorder
4.Preorder
5.Postorder
6.Count Leaf Nodes
7.Count Internal Nodes
8.Height of Tree
9.Exit
Enter choice: 4
2 1 4 3 6

WEEK – 11

PROGRAM-11: Binary Heap (Max Heap and Min Heap)

CODE:

```
class Heap:
    def __init__(self):
        self.heap = []
        self.type = ""

    def insert(self,data):
        self.heap.append(data)
        i = len(self.heap)-1

        while i > 0:
            p = (i-1)//2

            if self.type == "max" and self.heap[i] > self.heap[p]:
                self.heap[i],self.heap[p] = self.heap[p],self.heap[i]
            elif self.type == "min" and self.heap[i] < self.heap[p]:
                self.heap[i],self.heap[p] = self.heap[p],self.heap[i]
            else:
                break

            i = p

    def delete(self):
        if len(self.heap) == 0:
```

```
        print("Heap is empty")
        return

    print("Deleted root:",self.heap[0])

    if len(self.heap) == 1:
        self.heap.pop()
        return

    self.heap[0] = self.heap.pop()
    i = 0
    n = len(self.heap)

    while True:
        left = 2*i+1
        right = 2*i+2
        largest = i

        if self.type == "max":
            if left < n and self.heap[left] > self.heap[largest]:
                largest = left
            if right < n and self.heap[right] > self.heap[largest]:
                largest = right

        else:
            if left < n and self.heap[left] < self.heap[largest]:
```

```
            largest = left
        if right < n and self.heap[right] < self.heap[largest]:
            largest = right

        if largest == i:
            break

        self.heap[i],self.heap[largest] = self.heap[largest],self.heap[i]
        i = largest

    def display(self):
        if len(self.heap) == 0:
            print("Heap is empty")
        else:
            print("Heap:",self.heap)


while True:
    print("\n1.Max Heap")
    print("2.Min Heap")
    print("3.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1 or ch == 2:
        h = Heap()
```

```
if ch == 1:
    h.type = "max"
else:
    h.type = "min"

while True:
    print("\n1.Insert")
    print("2.Delete Root")
    print("3.Display")
    print("4.Back to Main Menu")

    op = int(input("Enter choice: "))

    if op == 1:
        data = int(input("Enter element: "))
        h.insert(data)

    elif op == 2:
        h.delete()

    elif op == 3:
        h.display()

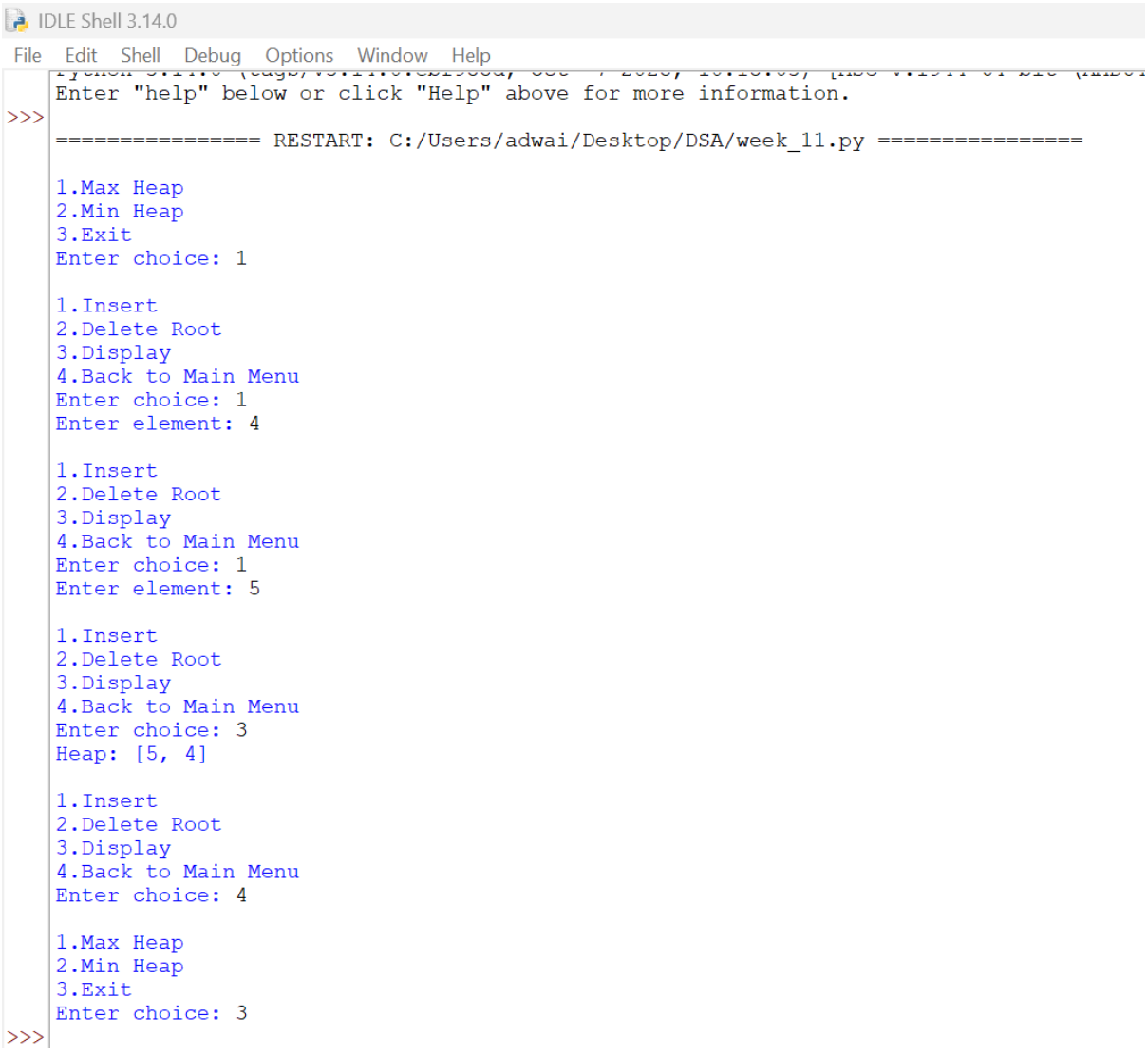
    elif op == 4:
        break
```

```
            else:
                print("Invalid choice")

    elif ch == 3:
        break

    else:
        print("Invalid choice")
```

OUTPUT:



WEEK – 12

PROGRAM-12:

Write a program to implement the following three collision resolution techniques in a Hash Table:

1. Separate Chaining
2. Linear Probing
3. Quadratic Probing

CODE:

1. Separate Chaining

```
class HashTable:
    def __init__(self,size):
        self.size = size
        self.table = [[] for i in range(size)]

    def insert(self,key):
        index = key % self.size
        self.table[index].append(key)

    def display(self):
        for i in range(self.size):
            print(i,":",self.table[i])

h = HashTable(10)

while True:
    print("\n1.Insert")
    print("2.Display")
    print("3.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        key = int(input("Enter key: "))
        h.insert(key)
    elif ch == 2:
        h.display()
    elif ch == 3:
        break
    else:
        print("Invalid choice")
```

OUTPUT:



CODE:

2. Linear Probing

```
class HashTable:
    def __init__(self,size):
        self.size = size
        self.table = [None]*size

    def insert(self,key):
        index = key % self.size

        for i in range(self.size):
            new_index = (index+i)%self.size

            if self.table[new_index] is None:
```

```
                self.table[new_index] = key
                return

        print("Hash Table is full")

    def display(self):
        for i in range(self.size):
            print(i,":",self.table[i])

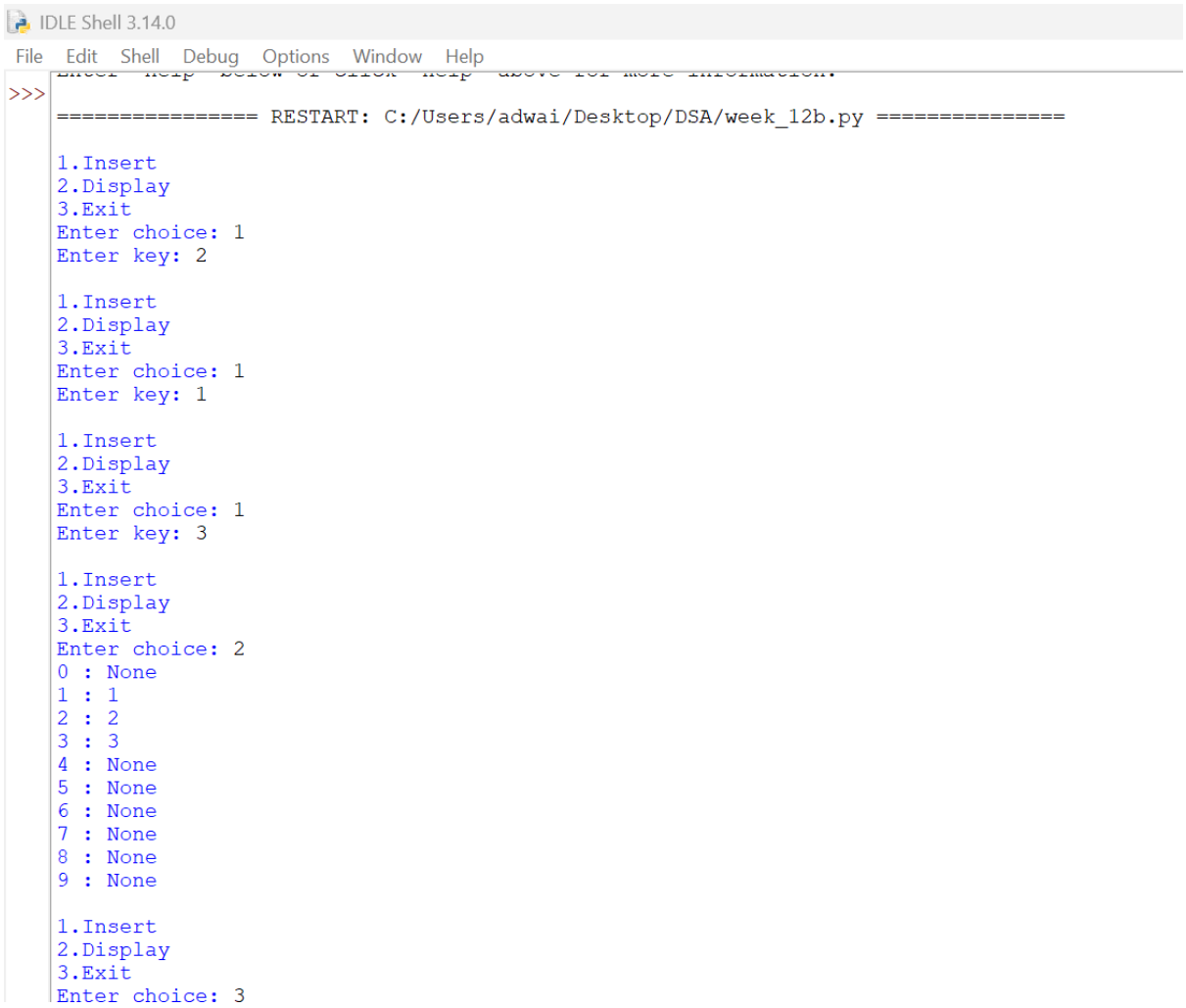
h = HashTable(10)

while True:
    print("\n1.Insert")
    print("2.Display")
    print("3.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        key = int(input("Enter key: "))
        h.insert(key)
    elif ch == 2:
        h.display()
    elif ch == 3:
        break
    else:
        print("Invalid choice")
```

OUTPUT:



CODE:

3. Quadratic Probing

```
class HashTable:
    def __init__(self,size):
        self.size = size
        self.table = [None]*size

    def insert(self,key):
        index = key % self.size

        for i in range(self.size):
            new_index = (index+i*i)%self.size

            if self.table[new_index] is None:
```

```
                self.table[new_index] = key
                return

        print("Cannot insert element")

    def display(self):
        for i in range(self.size):
            print(i,":",self.table[i])

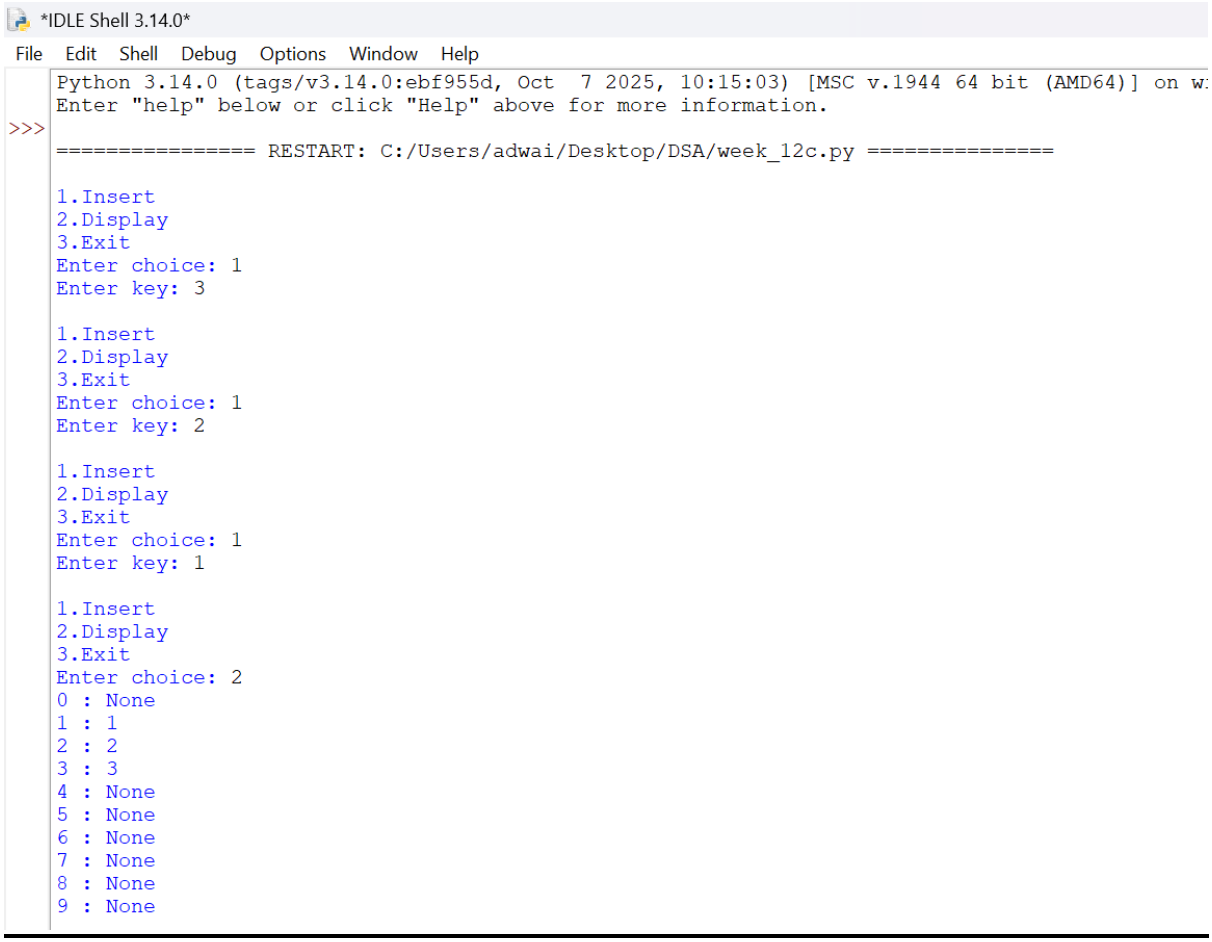
h = HashTable(10)

while True:
    print("\n1.Insert")
    print("2.Display")
    print("3.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        key = int(input("Enter key: "))
        h.insert(key)
    elif ch == 2:
        h.display()
    elif ch == 3:
        break
    else:
        print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on w
Enter "help" below or click "Help" above for more information.
>>>
================ RESTART: C:/Users/adwai/Desktop/DSA/week_12c.py ===============
1.Insert
2.Display
3.Exit
Enter choice: 1
Enter key: 3
1.Insert
2.Display
3.Exit
Enter choice: 1
Enter key: 2
1.Insert
2.Display
3.Exit
Enter choice: 1
Enter key: 1
1.Insert
2.Display
3.Exit
Enter choice: 2
0 : None
1 : 1
2 : 2
3 : 3
4 : None
5 : None
6 : None
7 : None
8 : None
9 : None